

Vue.js 3 フロントエンド開発入門

WSC0134G-L1-02

~作って理解する SPA(TypeScript 対応)~

演習ガイド

商標

- アマゾン ウェブ サービス、AWS および Amazon Web Services ロゴは、米国その他の諸国における、Amazon.com, Inc.またはその関連会社の商標です。
- Cisco、Cisco Systems、Cisco Systems ロゴは、Cisco Systems, Inc.および関連会社の米国ならびに他の国における商標もしくは登録商標です。
- Intel、インテルは、米国およびその他の国における Intel Corporation の商標です。
- ITIL®は AXELOS Limited の登録商標であり、AXELOS Limited の許可のもとに使用されています。すべての権利は留保されています。
- Microsoft、Windows は、米国 Microsoft Corporation の、米国およびその他の国における登録商標または商標です。
- Oracle と Java は、Oracle Corporation およびその子会社、関連会社の米国およびその他の国における登録商標です。
- PMI、PMP、PMBOK は、プロジェクトマネジメント協会(Project Management Institute, Inc.)の登録商標です。
- VMware、VMware ロゴは、VMware, Inc.の米国および各国での商標または登録商標です。
- BOOT CAMP、NEW TRAIN、MIX TRAIN、TRAINOCAMP は、トレノケート株式会社の登録商標です。

その他の会社名、製品名およびサービス名は、それぞれ各社の商標または登録商標です。

目次

演習 1 Vue3 環境構築(Vite)	5
演習 1.1. 開発環境の準備	7
演習 1.2. Vite でプロジェクト作成.....	7
演習 1.3. 依存ライブラリ導入	7
演習 1.4. 開発サーバーの起動と動作確認.....	8
演習 1.5. フォルダと主要ファイルの役割	8
演習 2 Vue.js の基礎知識.....	9
演習 2.1. Vue.js の特徴とメリット(クイズ)	11
演習 2.2. Vue コンポーネントの基本	11
演習 2.3. リアクティブなデータ管理.....	12
演習 2.3.1. ref() 関数(勤怠状況の管理).....	12
演習 2.3.2. reactive() 関数(社員情報の管理)	13
演習 2.3.3. computed()関数(手取り額の計算).....	14
演習 2.4. ディレクティブ構文.....	16
演習 2.4.1. v-bind ディレクティブ(画像・クラスの動的設定)	16
演習 2.4.2. v-if / v-else / v-show ディレクティブ	18
演習 2.4.3. v-for ディレクティブ(リスト表示).....	19
演習 2.5. イベント処理(v-on)	20
演習 2.6. フォーム入力と双方向バインディング (v-model)	21
演習 3 コンポーネント分割とデータの受け渡し	23
演習 3.1. コンポーネントの分割と配置.....	25
演習 3.2. 親から子へ値を渡す(props).....	27

演習 3.3. 【オプション】子から親へイベントを送る	29
演習 3.4. 複数の子コンポーネントを配置する	31
演習 4 ルーティングと SPA	33
演習 4.1. Vue Router の導入と役割(クイズ)	35
演習 4.2. ページ (ルート) とコンポーネント (クイズ).....	35
演習 4.3. ルート設定によるページ切り替え.....	36
演習 4.4. ナビゲーションメニューの作成	39
演習 4.5. Provide と inject によるデータ共有	41
演習 4.6. パラメータ付きルート(詳細画面の実装)	43
演習 5 【オプション】REST API 連携による CRUD 実装.....	45
演習 5.1. 社員一覧の表示	48
演習 5.2. 社員詳細の表示と削除	51
演習 5.3. 社員情報の編集	53
演習 5.4. 新規社員の登録	55
付録 A 選択式問題の解答と解説	57

演習 1 Vue3 環境構築 (Vite)

演習の目標

- Vite で Vue3 の開発環境を作成できる
- Vue3 プロジェクトのファイル構成を理解できる

演習の概要

第 1 章で登場した重要単語の意味を確認します。

予想所要時間

5 分

演習1.1. 開発環境の準備

Vuejs の開発環境準備 (Node.js と VS Code) に関する記述として、誤っているものを選んでください。

- a) VS Code で複数の章 (フォルダ) をまとめて管理するために、「ワークスペース」機能を利用すると便利である。
- b) Node.js をインストールすると、ブラウザ上で Java のコードが実行できるようになる。
- c) VS Code には、Vue ファイルの構文ハイライトや補完を有効にするために「Vue - Official」拡張機能をインストールする。
- d) Vue 3 や Vite を利用するためには、PC に Node.js がインストールされている必要がある。

演習1.2. Vite でプロジェクト作成

Vite を使用したプロジェクト作成の手順に関する記述として、誤っているものを選んでください。

- a) コマンド `npm create vite@latest` を使用して、最新の Vite プロジェクトを作成できる。
- b) コマンド実行時のディレクトリ位置 (カレントディレクトリ) にプロジェクトが展開されるため、実行場所には注意が必要である。
- c) プロジェクト作成コマンドを実行した直後、自動的に依存ライブラリ (`node_modules`) もインストールされるため、`npm install` は不要である。
- d) コマンドのオプションに `--template vue-ts` を指定することで、最初から TypeScript 対応の Vue プロジェクトを作成できる。

演習1.3. 依存ライブラリ導入

依存ライブラリの導入 (`npm install`) と `package.json` に関する記述として、誤っているものを選んでください。

- a) `npm install` を実行すると、`package.json` の内容に基づいて必要なファイルがインターネットからダウンロードされる。
- b) ダウンロードされたライブラリの実体は、`node_modules` フォルダの中に格納される。
- c) `package.json` は、プロジェクトに必要なライブラリの名前やバージョンが記述された設計図のようなファイルである。
- d) `npm install` を実行するためには、PC がインターネットに接続されている必要はない。

演習1.4. 開発サーバーの起動と動作確認

開発サーバーの起動と動作確認に関する記述として、誤っているものを選んでください。

- a) npm run dev コマンドを実行すると、ローカル開発サーバーが起動する。
- b) VS Code の「NPM スクリプト」機能を使うと、コマンド入力をせずにマウス操作だけでサーバーを起動できる。
- c) 開発サーバー起動中は、ソースコードを編集して保存すると、ブラウザをリロードしなくても変更が即座に反映される(ホットリロード)。
- d) 開発サーバーを停止すると、作成したソースコードもすべて削除されてしまう。

演習1.5. フォルダと主要ファイルの役割

Vue プロジェクトのフォルダ構成と主要ファイルの役割について、誤っているものを選んでください。

- a) App.vue はルートコンポーネントと呼ばれ、画面全体のレイアウトや子コンポーネントを統括する親の役割を持つ。
- b) index.html はアプリケーションのエントリーポイントであり、Vue アプリがマウントされる `<div id="app">` が記述されている。
- c) src/components フォルダには、ビルド後の成果物や画像ファイルなどの静的アセットのみを格納する。
- d) main.ts は TypeScript のエントリーポイントであり、`createApp(App).mount('#app')` を実行して Vue アプリを起動する。

演習 2 Vue.js の基礎知識

演習の目標

- Vue 3 の特徴とメリットを理解できる
- Vue コンポーネントの基本構文(<script setup>)を記述できる
- リアクティブなデータ管理(ref, reactive, computed)を実装できる
- ディレクティブ(v-bind, v-if, v-for)を使用して動的な画面を構築できる
- イベントハンドリング(v-on)と双方向バインディング(v-model)を実装できる

演習の概要

第 2 章で学習した Vue.js の基本文法を確認します。

プロジェクト名: ex02-question

理論的なセクションはクイズ形式で確認し、実践的なセクションはコンポーネントを作成して動作を確認します。

App.vue のナビゲーションボタンを使用して、該当する演習の画面に切り替えて確認してください。

- 社員情報の表示(データバインディング)
- 社員リストの表示(リストレンダリング)
- 給与情報の計算(算出プロパティ)
- 社員情報の編集機能(イベントと双方向バインディング)

予想所要時間

約 1 時間

演習2.1. Vue.js の特徴とメリット(クイズ)

従来の JavaScript (DOM 直接操作) の課題や、Vue.js の特徴に関する記述として、誤っているものを 1 つ選んでください。

- a) 従来の JavaScript 開発では、データの変更に合わせて手動で DOM 操作を行う必要があり、規模が大きくなるとコードが複雑化しやすいという課題があった。
- b) Vue.js は「リアクティブシステム」を備えており、JavaScript 側のデータが変更されると、自動的に画面 (DOM) の表示も更新される。
- c) Vue.js はサーバーサイド (バックエンド) 専用のフレームワークであり、Java や PHP のようにサーバー上で HTML を生成してブラウザに送信する仕組みである。
- d) Vue.js では、HTML・CSS・JavaScript を「単一ファイルコンポーネント(.vue)」として 1 つのファイルにまとめて管理し、部品化することができる。

演習2.2. Vue コンポーネントの基本

修正対象ファイル: src/components/Q2_2.vue

「2.2」のボタンをクリックすると、Q2_2.vue のコードに切り替えできる設定済みです。

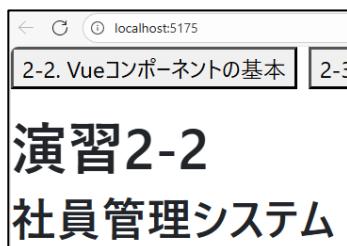
a. タイトル変数の定義

- script setup 内で、定数 systemName を定義し、値 '社員管理システム' を代入してください。

b. 画面への表示

- template 内で、マスタッシュ構文 {{ }} を使って systemName を h2 タグで表示してください。

動作確認



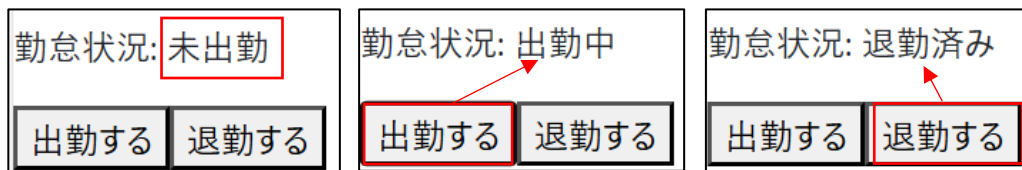
- ブラウザで「2-2. Vue コンポーネントの基本」をクリックし、システム名が表示されることを確認してください。

演習2.3. リアクティブなデータ管理

社員のデータや状態を管理するための変数を定義します。

演習2.3.1. ref() 関数(勤怠状況の管理)

単一の値(プリミティブな値)をリアクティブにする練習として、出勤・退勤の状態を管理します。



修正対象ファイル: src/components/Q2_3_1.vue

a. 変数の定義

- vue から ref をインポートしてください。
- 文字列型の変数 workState(初期値 '未出勤')を ref で定義してください。

b. 更新関数の作成

- 以下の2つの関数を作成してください。
 - workStart: workState の値を '出勤中' に変更する。戻り値無し
 - workEnd: workState の値を '退勤済み' に変更する。戻り値無し
- ※ script 内で ref の値を操作する際は .value が必要な点に注意してください。

c. 画面表示

- template に以下の要素を配置してください。
 - 見出し: <h1>演習 2-3-1</h1>
 - 現在の状態: <p>勤怠状況: (workState を表示するコード) </p>
 - ボタン: 「出勤する」(クリックで workStart を実行)
 - ボタン: 「退勤する」(クリックで workEnd を実行)

動作確認

- 「2-3-1. ref()関数」画面で、各ボタンを押して勤怠状況の表示が変わるか確認してください。

演習2.3.2. reactive() 関数（社員情報の管理）

オブジェクトをまとめてリアクティブにする練習として、「社員情報」を管理します。

修正対象ファイル: `src/components/Q2_3_2.vue`

a. オブジェクトの定義

- vue から `reactive` をインポートしてください。
- `reactive` を使い、以下のプロパティを持つ社員情報オブジェクト `employee` を定義してください。
 - `name`: '田中 太郎'
 - `department`: '開発部'
 - `workLocation`: '未設定'

b. 更新関数の作成

- 以下の 2 つの関数を作成してください。
 - `changeOfficeWork`: `workLocation` を '新宿オフィス' に変更する。戻り値無し
 - `changeRemoteWork`: `workLocation` を '在宅勤務' に変更する。戻り値無し
- ※ `reactive` のプロパティにアクセスする際は `.value` は不要です。

c. 画面表示

- `template` に以下の情報をオブジェクトから取り出して表示してください。
 - 氏名:
 - 部署:
 - 出勤場所:
- 「オフィス」ボタンと「リモートワーク」ボタンに正しい関数を入れてください。

動作確認

「2-3-2. reactive()関数」画面で、ボタンを押して出勤場所が変わるか確認してください。

演習2.3.3. computed()関数(手取り額の計算)

既存のデータから新しい値を計算する練習として、税金(20%)を引いた手取り額を自動計算します。

修正対象ファイル: src/components/Q2_3_3.vue

a. データ準備

- reactive の社員情報 employee に salary を追記している部分を確認してください。
 - name: '田中 太郎'
 - department: '開発部'
 - workLocation: '未設定'
 - salary: 250000 (月給)

b. 手取り額の算出

- ① vue から computed をインポートしてください。
- ② 給与の 80% を手取り額として計算する算出プロパティ salaryTaxPlus を定義してください。
 - 計算式: $\text{employee.salary} * 0.8$

c. 画面表示

- ① template に以下の要素を追記してください。
 - 月給: (社員の税抜きの給与を表示) 円
 - 推定手取り額: (税込み給与を表示) 円

- ② クリックすると `employee.salary` が 10000 増える「昇給」ボタンのソースコードを確認してください。

動作確認

- 「2-3-3. `computed()`関数」画面で、昇給ボタンを押すと月給が増え、それに連動して手取り額も自動的に再計算されることを確認してください。

演習2.4. ディレクティブ構文

HTML の属性や表示を動的に操作する方法を学びます。

演習2.4.1. v-bind ディレクティブ（画像・クラスの動的設定）

社員の出勤場所によって、プロフィール画像および文字色が変化するように実装します。



修正対象ファイル: src/components/Q2_4_1.vue

a. データの定義(演習 2.3.2 のソースコードに追記)

- reactive を使い、employee オブジェクトを定義してください(プロパティは以下の通り)。
 - name: '田中 太郎'
 - department: '開発部'
 - workLocation: '未設定'
 - photoUrl: '/images/photo1.png' ←追記
 - textColor: 'black' ←追記

b. 更新関数の実装

- changeOfficeWork 関数に以下のように追記してください。:
 - 勤務地を '新宿オフィス' に変更
 - 写真を '/images/photo2.png' に変更
 - 文字色クラスを 'text-primary'(青色)に変更
- changeRemoteWork 関数に以下のように追記してください。:
 - 勤務地を '在宅勤務' に変更
 - 写真を '/images/photo3.png' に変更
 - 文字色クラスを 'text-danger'(赤色)に変更

c. 画面表示(v-bind の適用)

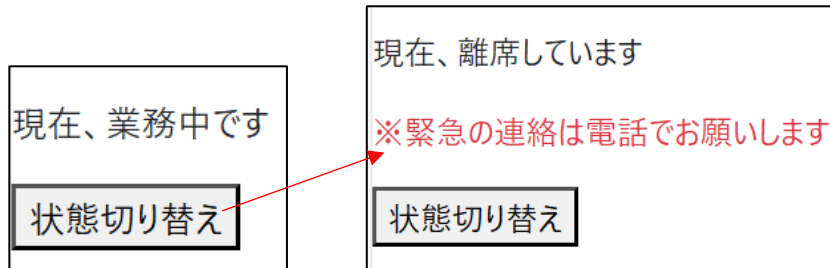
- template 内の指示コメントに従って、以下の要素に v-bind(または :)を適用してください。
 - img タグの src 属性 → employee.photoUrl
 - 出勤場所を表示する p タグの class 属性 → employee.textColor

動作確認

- 「2-4-1. v-bind」画面で、ボタンを押すと「画像」と「出勤場所」の文字色が同時に切り替わることを確認してください。

演習2.4.2. v-if / v-else / v-show ディレクティブ

社員のステータス(在席中かどうか)に応じて表示メッセージを切り替えます。



修正対象ファイル: src/components/Q2_4_2.vue

a. ステータス変数の定義

- boolean 型の変数 `isOnline`(初期値 `true`)を `ref` で定義してください。

b. 切り替え関数の実装

- `changeWorkStatus` 関数を作成し、`isOnline` の値を反転(`true` ⇔ `false`)させてください。

c. 表示の出し分け(v-if と v-else)

- `v-if` / `v-else` を使い、以下の表示を切り替えてください。
 - `isOnline` が `true` の場合: 「現在、業務中です」
 - `isOnline` が `false` の場合: 「現在、離席しています」

d. 補足メッセージの表示(v-show)

- `v-show` ディレクティブを使い、`isOnline` が `false`(離席中)の時だけ表示される以下のメッセージを追加してください。
 - メッセージ: 「※緊急の連絡は電話をお願いします」

動作確認

- 「2-4-2. v-if」画面で、「状態切り替え」ボタンを押すとメッセージが切り替わり、離席中の時だけ赤字の補足メッセージが表示されることを確認してください。

演習2.4.3. v-for ディレクティブ (リスト表示)

複数の社員データを配列として管理し、v-for ディレクティブを使ってテーブル形式で一覧表示します。

ID	氏名	年収	部署
101	佐藤一郎	3500000 円	営業部
102	鈴木二郎	3500000 円	総務部
103	高橋三郎	5000000 円	開発部
104	田中士郎	5500000 円	営業部
105	伊藤五郎	4800000 円	総務部

修正対象ファイル: `src/components/Q2_4_3.vue`

a. 社員リストの定義

- 以下のオブジェクトを含む配列 `employeeList` を定義している `reactive` 部分を確認してください。
 - `{ id: 101, name: '佐藤一郎', salary: 3500000, department: '営業部' }`
 - `{ id: 102, name: '鈴木二郎', salary: 3500000, department: '総務部' }`
 - `{ id: 103, name: '高橋三郎', salary: 5000000, department: '開発部' }`
 - `{ id: 104, name: '田中四郎', salary: 5500000, department: '営業部' }`
 - `{ id: 105, name: '伊藤五郎', salary: 4800000, department: '総務部' }`

b. テーブルの作成(v-for の使用)

- `template` 内の `<table>` と `<thead>` のソースコードを確認してください。
- `tbody` 内の `tr` タグに `v-for` を適用し、`employeeList` の各要素を表示してください。
 - 重要: `:key` 属性には必ず一意な `id` を指定してください。

c. 動作確認

- 「2-4-3. v-for」画面で、社員リストがテーブル形式で表示されることを確認してください。

演習2.5. イベント処理(v-on)

テーブル内のボタンをクリックした際に、その行のデータを引数としてメソッドに渡し、処理を行う機能を実装します。

ID	氏名	年収	操作
101	佐藤一郎	3500000 円	選択
102	鈴木二郎	3500000 円	選択
103	高橋三郎	5000000 円	選択
104	田中士郎	5500000 円	選択
105	伊藤五郎	4800000 円	選択

選択した社員：高橋三郎			
ID	氏名	年収	操作
101	佐藤一郎	3500000 円	選択
102	鈴木二郎	3500000 円	選択
103	高橋三郎	5000000 円	選択
104	田中士郎	5500000 円	選択
105	伊藤五郎	4800000 円	選択

修正対象ファイル: src/components/Q2_5.vue

a. データの準備

- 選択された社員情報を表示するための変数 `selectedInfo` (初期値 "") を `ref` で定義してください。
- `employeeList` は演習 2.4.3 と同様のデータを使います。

b. 選択メソッドの実装(引数あり)

- `selectEmployee(name)` 関数を作成してください。引数として「氏名」受け取ります。
- 関数内で、`selectedInfo.value` に 選択した社員: `${name}` という文字列を代入してください。
- ※文字列と変数を並べる時は、バッククォート(`) で囲って、`社員 \${変数名}` の書き方で埋め込みます。

c. テーブルへのボタン追加

- `thead` に「操作」列を追加してください。
- `tbody` の各行に「選択」ボタンを追加し、`@click` イベントを設定してください。
- `@click` では `selectEmployee` 関数を呼び出し、引数として現在の行の `emp.name` を渡してください。

d. 結果の表示

- テーブルの上に `<p>` タグを作成し、`selectedInfo` の内容を表示してください。

動作確認

- 「2-5. イベント処理」画面で、「選択」ボタンをクリックすると、その行の社員名と年収が画面下部に表示されることを確認してください。

演習2.6. フォーム入力と双方向バインディング (v-model)

さまざまなフォーム要素(テキスト、セレクトボックス、ラジオボタン、チェックボックス)を使って、社員登録フォームを作成します。入力した内容は、リアルタイムで HTML にも反映される画面を作成します。

The image shows two side-by-side screenshots of a form. The left screenshot shows the initial state: '氏名' is empty, '部署' is '営業部', '雇用形態' has '正社員' selected, and '所持スキル' has 'Java', 'Vue.js', and 'Python' unchecked. The right screenshot shows the state after a change: '氏名' is '田中太郎', '部署' is '開発部', '雇用形態' has '契約社員' selected, and '所持スキル' has 'Java' and 'Vue.js' checked. A red box highlights the input fields in the right screenshot, and a red arrow points from the '開発部' dropdown to the '登録内容プレビュー' section below.

修正対象ファイル: src/components/Q2_6.vue

a. フォームデータの定義

- reactive を使い、以下のプロパティを持つオブジェクト form のソースコードを確認してください。
 - name: '' (氏名)
 - department: '未設定' (部署)
 - employmentType: '正社員' (雇用形態)
 - skills: [] (所持スキル - 配列)

b. 入力フォームの作成

- 以下のフォーム要素を作成し、v-model で form の各プロパティとバインドしてください。

1. 氏名: `<input type="text">`
2. 部署: `<select>` タグ(選択肢: 営業部, 開発部, 総務部)
3. 雇用形態: 3 つの `<input type="radio">`(選択肢: 正社員, 契約社員, パート)
 - ※同じ `name` 属性を付けるか、同じ `v-model` を指定することでグループ化されます。
4. 所持スキル: 3 つの `<input type="checkbox">`(選択肢: Java, Vue.js, Python)
 - ※ `value` 属性にスキル名を設定し、`v-model` で同じ配列 (`form.skills`) にバインドします。

c. 入力内容の確認表示

- フォームの下に区切り線(`<hr>`)を引き、入力された値をリアルタイムで表示するエリアを作成してください。

動作確認

- 「2-6. フォーム入力」画面で、フォームを操作すると下の表示が即座に変わることを確認してください。特にチェックボックスは、選択した項目が配列として増減することを確認してください。

演習 3 コンポーネント分割とデータの受け渡し

演習の目標

- 画面を機能ごとの部品(コンポーネント)に分割して構築できる
- `import` 文を使用して子コンポーネントを読み込み、`template` 内で使用できる
- 親コンポーネントから子コンポーネントへデータ(`props`)を渡すことができる
- 複数の子コンポーネントを持つ画面を実装できる
- (オプション)子コンポーネントから親コンポーネントへイベント(`emit`)を通知できる

演習の概要

第 3 章で学習したコンポーネント指向の開発手法を実践します。

プロジェクト名: `ex03-question`

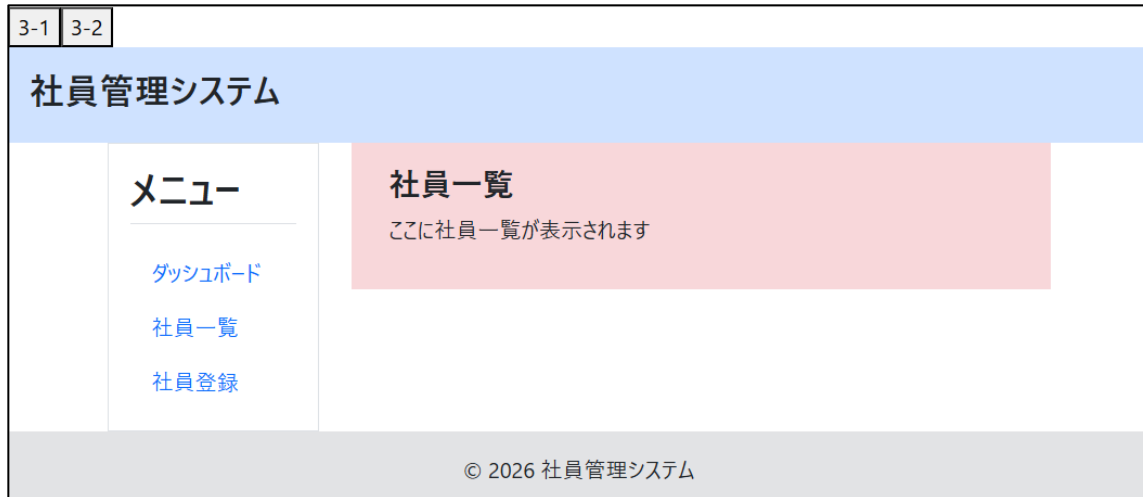
社員管理システムの画面を複数の部品に分割し、データの受け渡しを行うことで、保守性の高いアプリケーション構造を体験します。App.vue のナビゲーションボタンを使用して、該当する演習の画面に切り替えて確認してください。

予想所要時間

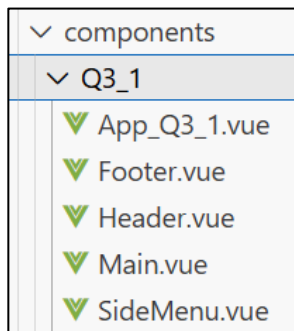
約 40 分

演習3.1. コンポーネントの分割と配置

画面を「ヘッダー」「サイドメニュー」「メイン」「フッター」の 4 つのコンポーネントに分割し、親コンポーネントで組み立ててレイアウトを作成します。



作成・修正するファイル:(演習問題ごと、フォルダ分けしています)



- 親: src/components/Q3_1/App_Q3_1.vue
- 子: src/components/Q3_1/Header.vue
- 子: src/components/Q3_1/SideMenu.vue
- 子: src/components/Q3_1/Main.vue
- 子: src/components/Q3_1/Footer.vue

a. 子コンポーネントの作成

src/components/Q3_1/ フォルダ内に以下の 4 つのファイルがあります。それぞれのコードを確認してください。

※テキストのサンプルと類似なデザインのソースコードを使います。

1. Header.vue

- <header> タグを使用し、クラス bg-primary-subtle p-3 を適用。
- タイトル「社員管理システム」を表示(クラス fs-3)。

2. SideMenu.vue

- <aside> タグを使用し、クラス bg-white border p-3 h-100 を適用。
- 「メニュー」見出しと、リスト(ダッシュボード、社員一覧、社員登録)を作成。

3. Main.vue

- <main> タグを使用し、クラス p-3 を適用。
- 仮の内容として「ここにメインコンテンツが表示されます」と記述。

4. Footer.vue

- <footer> タグを使用し、クラス bg-secondary-subtle p-3 text-center を適用。
- コピーライト「© 2026 社員管理システム」を表示。

b. 親コンポーネントでの組み立て (App_Q3_1.vue)

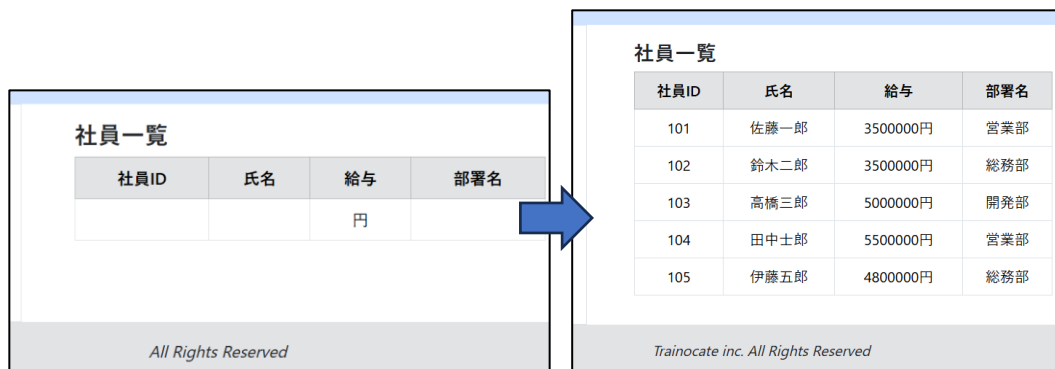
- ① script setup で上記 4 つのコンポーネントをインポートしてください。
- ② template では Bootstrap のグリッドシステム(container, row, col-md-3, col-md-9)を使用してレイアウトを組んでいます。コメントで指示している場所に、各コンポーネントを配置してください。

動作確認

- ブラウザで「3-1」ボタンをクリックし、ヘッダー・サイドバー・メイン・フッターが正しく配置されていることを確認してください。

演習3.2. 親から子へ値を渡す(props)

親コンポーネントで管理している「会社名」や「社員リスト」などのデータを、Props（プロップス）という仕組みを使って子コンポーネントへ渡し、画面に表示させます。



作成・修正するファイル:

- 親: src/components/Q3_2/App_Q3_2.vue
- 子: src/components/Q3_2/Footer.vue (単一データを送信)
- 子: src/components/Q3_2/Main.vue (配列データを送信)
- ※その他の子コンポーネント (Header, SideMenu) は Q3_1 のものをコピーして Q3_2 フォルダに配置しています。

a. 親コンポーネントの実装 (App_Q3_2.vue)

src/components/Q3_2/App_Q3_2.vue を編集します。

- ① script setup 内で、以下のデータを定義している部分を確認してください。
 - 定数 companyName: 値 'Trainocate inc.'
 - リアクティブな配列 employeeList: 5 名分の社員オブジェクト (id, name, salary, department)
- ② template 内で <Main> タグと <Footer> タグを配置し、v-bind(:) を使って定義した変数をそれぞれのコンポーネントに渡してください。

b. 子コンポーネントの作成 (Footer.vue)

src/components/Q3_2/Footer.vue を編集します。

- ① script setup 内で、受け取るデータの型 FooterProps (companyName: 文字列) を定義してください。

- ② `defineProps` を使用して、定義した型の `Props` を受け取れるようにしてください。
- ③ `template` 内でコピーライトの会社名部分が表示されるように記述してください。

c. 子コンポーネントの作成(Main.vue)

`src/components/Q3_2/Main.vue` を編集します。

- ① `script setup` 内で、社員 1 人分の型 `EmployeeList(id, name, salary, department)` を定義してください。
- ② `defineProps` を使用して、`employeeList` という名前で **`EmployeeList` の配列** を受け取れるようにしてください。
- ③ `template` 内の `tr` タグに `v-for` を適用し、受け取ったリストの中身(社員 ID、氏名、給与、部署名)をテーブルに表示してください。

動作確認

- ブラウザで「3-2」ボタンをクリックしてください。
- メインエリアに、定義した 5 名分の社員リストがテーブル形式で表示されていることを確認してください。
- フッターエリアに、「Trainocate inc. All Rights Reserved」と会社名が表示されていることを確認してください。

演習3.3. 【オプション】子から親へイベントを送る

※3.4 まで早く終わった方は、こちらに挑戦してください。

子コンポーネント(社員一覧)にあるボタンをクリックした際、その情報を親コンポーネントへ通知(emit)し、親側で受け取ったデータを画面に表示させます。

選択中の社員:				
社員一覧				
社員ID	氏名	給与	部署名	操作
101	佐藤一郎	3500000円	営業部	選択
102	鈴木二郎	3500000円	総務部	選択

選択中の社員: 鈴木二郎				
社員一覧				
社員ID	氏名	給与	部署名	操作
101	佐藤一郎	3500000円	営業部	選択
102	鈴木二郎	3500000円	総務部	選択

作成・修正するファイル:

- 親: src/components/Q3_3/App_Q3_3.vue
- 子: src/components/Q3_3/Main.vue
- ※その他の子コンポーネント(Header, SideMenu, Footer)は Q3_2 のものをコピーしてあります。
-

a. 子コンポーネントの実装(Main.vue)

src/components/Q3_3/Main.vue を編集します。

- script setup 内で、defineEmits を使用して、親へ通知するためのイベント(イベント名: 'select')を定義してください。
- 社員が選択された時に実行する関数(例: onSelect)を作成してください。この関数内で emit を実行し、引数として受け取った「社員名」を親へ渡します。
- template のテーブル見出し- テーブルのデータ行

b. 親コンポーネントの実装(App_Q3_3.vue)

src/components/Q3_3/App_Q3_3.vue を編集します。

- script setup 内で、選択された社員名を保持するためのリアクティブ変数(例: selectedEmployeeName)を ref で定義してください(初期値は空文字 '')。
- 子から通知を受け取った時に実行する関数(例: handleSelect)を作成し、引数で受け取った名前をリアクティブ変数に代入してください。

- ③ template 内の <Main> タグにイベントリスナ(@select="...")を追加し、作成した関数と紐付けてください。
- ④ 画面の右上に、選択された社員名を表示するエリア(選択中の社員: ...)を追加してください

動作確認

- ブラウザで「3-3」ボタンをクリックしてください。(※App.vue 等で切り替え設定をしている場合)
- 社員一覧の右側にある「選択」ボタンをクリックしてください。
- クリックした社員の名前が、画面右上に「選択中の社員: ○○」として表示されることを確認してください。

演習3.4. 複数の子コンポーネントを配置する

v-for ディレクティブを使用して、コンポーネント自体を繰り返し表示させる機能を実装します。これまでは「Main コンポーネント」内でテーブルを使って社員一覧を表示していましたが、今回は Main コンポーネントを廃止し、App コンポーネントから直接「社員カードコンポーネント」を繰り返し呼び出す構成に変更します。また、社員データには顔写真を追加し、より見やすい一覧画面を作成します。



作成・修正するファイル:

- 親: src/components/Q3_4/App_Q3_4.vue
- 子(新規): src/components/Q3_4/EmployeeCard.vue
- ※その他の子コンポーネント (Header, SideMenu, Footer) は Q3_2 のものをコピーして Q3_4 フォルダに配置してあります

変更点のイメージ

- Before (演習 3.2/3.3):
 - App → Main(テーブル表示)
- After (本演習):
 - App → EmployeeCard(カード表示) × 人数分
 - ※Main コンポーネントを経由せず、App から直接カードを並べる構成に変わります。

a. 子コンポーネントの作成(EmployeeCard.vue)

src/components/Q3_4/EmployeeCard.vue を編集します。

- ① script setup 内で、社員 1 名分の型定義(Employee)を行っていますので、確認してください。
- ② 親から employee という名前でデータを受け取るように defineProps を記述してください。
- ③ template 内で Bootstrap の card クラスを使用し、以下の要素を配置してください。
 - 顔写真(img タグ):src 属性に props.employee.photoUrl をバインド
 - 社員名
 - 部署名
 - フッター部分に社員 ID

b. 親コンポーネントの実装(App_Q3_4.vue)

src/components/Q3_4/App_Q3_4.vue を編集します(Q3_2 をベースに変更)。

- ① **データの修正:** employeeList の各オブジェクトに、photoUrl: '/images/101.png' などの画像パスを追加している部分を確認してください。
- ② **テンプレートの変更:**
 - <Main> コンポーネントを削除します。
 - 代わりに、v-for を使って <EmployeeCard> コンポーネントを繰り返し表示するように追記してください。属性名は employee にしてください。
 - グリッドシステム(col-md-3)を使用し、カードが横並び(例:1 行に 4 枚)になるよう調整してください

動作確認

- ブラウザで「3-4」ボタンをクリックしてください。
- 従来のテーブル形式ではなく、顔写真付きのカードがグリッド状に並んで表示されていることを確認してください。

演習 4 ルーティングと SPA

演習の目標

- Vue Router を導入して、ページ遷移の仕組みを理解できる
- ルートとコンポーネントを対応づけて表示を切り替えられる
- メニュー操作でページを動的に切り替える SPA 構成を実装できる
- ページ間のデータ共有ができる
- ルートパラメータを使って、ページ間で値を受け渡せる

演習の概要

第 4 章で学習した Vue Router を利用した SPA (シングルページアプリケーション) の開発手法を実践します。

プロジェクト名: ex04-question

URL によって表示するコンポーネントを切り替える「ルーティング」の設定を行い、従来の画面遷移 (ページ再読み込み) を行わずに画面が切り替わるアプリケーションを構築します。

予想所要時間

約 40 分

演習4.1. Vue Router の導入と役割(クイズ)

Vue Router の概要と導入に関する記述として、誤っているものを選んでください。

- a) Vue Router を使用すると、URL に応じて表示するコンポーネントを切り替えることができ、SPA (Single Page Application)として動作させることができる
- b) Vue Router を既存のプロジェクトに導入する場合、ターミナルで `npm install vue-router` コマンドを実行してライブラリを追加する。
- c) Vue Router を使用するためには、`main.ts` に `.use(router)` の記述を追加し、アプリ全体でルーターを有効化する必要がある。
- d) Vue Router は Vue.js 本体に最初から組み込まれている機能であるため、別途インストールや `package.json` への追加は不要である。

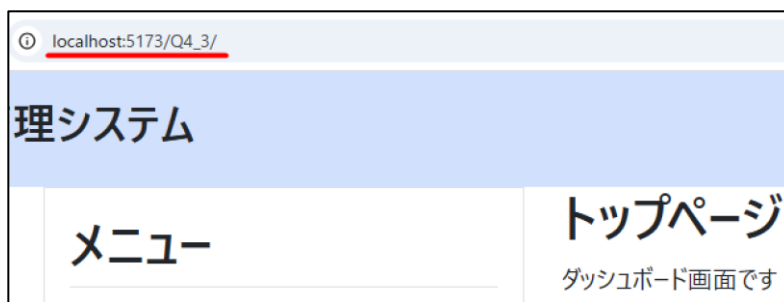
演習4.2. ページ (ルート) とコンポーネント (クイズ)

SPA における画面構成と <RouterView> の役割に関する記述として、誤っているものを選んでください。

- a) SPA では、ヘッダーやサイドバーなどの共通部分は `App.vue` に固定して配置し、メインコンテンツ部分のみを切り替える構成が一般的である。
- b) <RouterView /> は、URL に対応したコンポーネント(ページ)が表示される「場所」を確保するためのタグである。
- c) ページを切り替えるたびにブラウザが HTML ファイル全体を再読み込みするため、動作は従来の Web サイトと同じように少し遅くなる。
- d) `src/views` フォルダには、ページとして表示されるコンポーネント(例: `HomeView.vue`)を格納し、`src/components` フォルダには部品コンポーネントを格納するのが推奨される構成である。

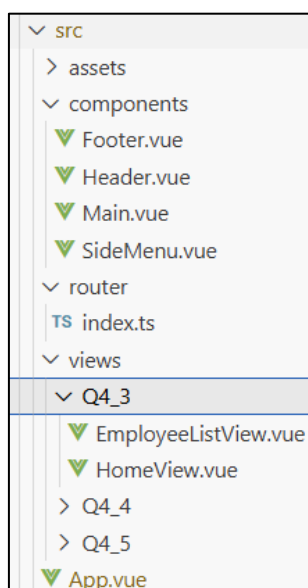
演習4.3. ルート設定によるページ切り替え

実際に Vue Router の設定を行い、URL に応じて「ダッシュボード(トップページ)」と「社員一覧ページ」が切り替わるように実装します。本演習からは、src/views フォルダ配下に演習用のフォルダを作成して管理します。



作業の前提:

- Vue-router モジュールは、npm から導入済みです。
- 3 章で作成した Header.vue, SideMenu.vue, Footer.vue が src/components 内に存在している状態から開始します。



- 4 章からは、views フォルダの中に演習ごとのフォルダを作成します。

作成・修正するファイル:

- **設定:** src/router/index.ts
- **画面:** src/views/Q4_3/HomeView.vue (新規作成)
- **画面:** src/views/Q4_3/EmployeeListView.vue (新規作成)
- **親:** src/App.vue

a. 画面コンポーネントの準備の確認

- src/views フォルダ内に Q4_3 フォルダを作成しました。
- その中に、以下の 2 つのファイルを作成しました。(まずは表示確認用として、簡易的なメッセージのみ記述します)
 1. HomeView.vue
 - template 内に <h2>ダッシュボード</h2><p>ダッシュボード画面です。</p> を記述。
 2. EmployeeListView.vue
 - template 内に <h2>社員一覧</h2><p>社員一覧画面です。</p> を記述。

b. ルーター設定ファイルの作成 (src/router/index.ts)

- src/router/index.ts を開き、以下の設定を記述(または修正)してください。
 - createRouter, createWebHistory をインポート。
 - ① 作成した HomeView.vue と EmployeeListView.vue をインポート。
 - ※インポート名は HomeViewQ4_3 のように末尾に演習番号をつけると区別しやすくなります。
 - ② routes 配列を定義し、以下のパスを設定します。
 - path: '/Q4_3' → component: HomeViewQ4_3
 - path: '/Q4_3/employees' → component: EmployeeListViewQ4_3

c. ルータービューの配置 (src/App.vue)

src/App.vue を編集します。

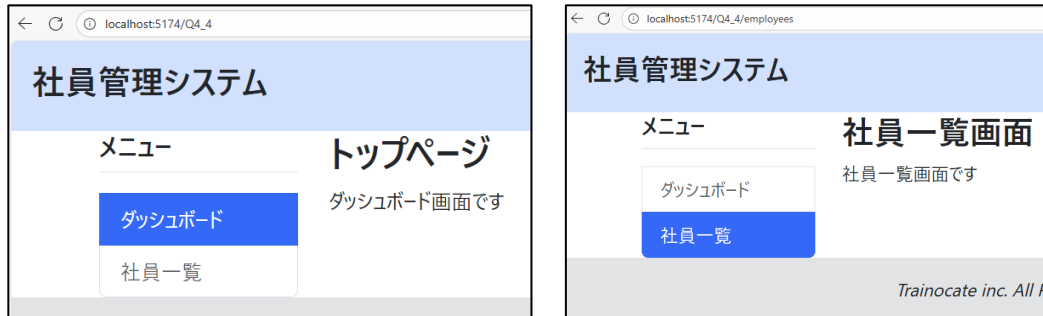
- Header, SideMenu, Footer はそのまま残します。
- ① Vue Router の RouterView コンポーネントをインポートしていることを確認してください。
- ② 3 章で配置していた <Main ... /> コンポーネント(およびインポート文)を削除またはコメントアウトしてください。
- ③ 代わりに、メインコンテンツが表示されるエリア(col-md-9 の div タグの中)に <RouterView /> タグを配置してください。

動作確認

- ex04 の開発サーバーを起動 (npm run dev) してください。localhost:517●の「●」の番号を確認します。
- ブラウザのアドレスバーで URL を直接書き換えて、ヘッダー等は固定されたまま、メイン部分だけが変わるか確認してください。
 - http://localhost:517●/Q4_3 → 「ダッシュボード画面です」と表示される
 - http://localhost:517●/Q4_3/employees → 「社員一覧画面です」と表示される

演習4.4. ナビゲーションメニューの作成

URL を直接打ち込むのではなく、サイドメニューのリンクをクリックして画面を切り替えられるようにします。Vue Router が提供する <RouterLink> コンポーネントを使用することで、ページの再読み込みを行わずに高速に画面を切り替えることができます。



修正するファイル:

- 部品: `src/components/SideMenu.vue`
- 設定: `src/router/index.ts`

a. RouterLink への置き換え

`src/components/SideMenu.vue` を編集します。

- ① <a> タグなどで書かれているメニュー項目を<RouterLink>タグに変更してください。
- ② to 属性には、以下のパスを指定します。
 - ・ダッシュボード: `to="/Q4_4"`
 - ・社員一覧: `to="/Q4_4/employees"`
- ③ Bootstrap のクラス (`list-group-item` 等) はそのまま維持してください。
- ④ クリックされたメニューをハイライト表示するために、<RouterLink> に `exact-active-class="active"` 属性を追加してください。

b. ルーター設定ファイルの作成 (`src/router/index.ts`)

- `src/router/index.ts` を開き、以下のように編集します。
 - ① `Q4_4` の `HomeView.vue` と `EmployeeListView.vue` をインポート。
 - ② `routes` 配列を定義し、以下のパスを設定します。
 - `path: '/Q4_4' → component: HomeViewQ4_4`
 - `path: '/Q4_4/employees' → component: EmployeeListViewQ4_4`

動作確認

- ブラウザで <http://localhost:517>●/Q4_4 にアクセスしてください。
- サイドメニューの「ダッシュボード」「社員一覧」をクリックし、ブラウザの更新ボタンが回ることなく、画面(メインエリア)がスムーズに切り替わることを確認してください。
- 選択中のメニューの背景色が濃くなり(active 状態)、現在地がわかるようになっているか確認してください。

演習4.5. Provide と inject によるデータ共有

App.vue で管理している「社員データ(employeeList)」を、provide を使ってアプリ全体に提供し、子画面である EmployeeListView で inject して受け取ります。今回はコンポーネントを使わず、受け取ったデータをそのまま **テーブル形式** で一覧表示します。

社員管理システム																											
メニュー		社員一覧																									
ダッシュボード		<table><thead><tr><th>社員ID</th><th>氏名</th><th>給与</th><th>部署</th></tr></thead><tbody><tr><td>101</td><td>佐藤一郎</td><td>3500000円</td><td>営業部</td></tr><tr><td>102</td><td>鈴木二郎</td><td>3500000円</td><td>総務部</td></tr><tr><td>103</td><td>高橋三郎</td><td>5000000円</td><td>開発部</td></tr><tr><td>104</td><td>田中土郎</td><td>5500000円</td><td>営業部</td></tr><tr><td>105</td><td>伊藤五郎</td><td>4800000円</td><td>総務部</td></tr></tbody></table>		社員ID	氏名	給与	部署	101	佐藤一郎	3500000円	営業部	102	鈴木二郎	3500000円	総務部	103	高橋三郎	5000000円	開発部	104	田中土郎	5500000円	営業部	105	伊藤五郎	4800000円	総務部
社員ID	氏名	給与	部署																								
101	佐藤一郎	3500000円	営業部																								
102	鈴木二郎	3500000円	総務部																								
103	高橋三郎	5000000円	開発部																								
104	田中土郎	5500000円	営業部																								
105	伊藤五郎	4800000円	総務部																								
社員一覧																											
Trainocate inc. All Rights Reserved																											

作成・修正するファイル:

- 設定: src/components/SideMenu.vue
- 親: src/App.vue
- 画面: src/views/Q4_5/EmployeeListView.vue

a. サイドメニューのリンク先の修正 (SideMenu.vue)

src/components/SideMenu.vue を修正します。

- ① RouterView to のリンク先を、Q4_4 から、全て Q4_5 に修正してください。

b. ルーター設定ファイルの作成 (src/router/index.ts)

- src/router/index.ts を開き、以下のように編集します。
 - ① Q4_5 の HomeView.vue と EmployeeListView.vue をインポート。
 - ② routes 配列を定義し、以下のパスを設定します。
 - path: '/Q4_5' → component: HomeViewQ4_5
 - path: '/Q4_5/employees' → component: EmployeeListViewQ4_5

c. データの提供 (App.vue)

src/App.vue を編集します。

- ② vue から provide をインポートしてください。
- ③ script setup 内で、定義済みの employeeList を、キー名 'employeeList' で provide してください。

d. データの注入と表示(EmployeeListView.vue)

- src/views/Q4_5/EmployeeListView.vue を編集します。
- スクリプト:
 - ① vue から inject をインポートしてください。
 - ② vue から Ref を import type してください。
 - ③ 社員データの型定義(Employee 型)を確認してください。(App.vue のデータ構造 id, name, salary, department と合わせる)。
 - ④ inject を使用して、親から提供された 'employeeList' を受け取り、変数(例: employeeList)に格納してください。
 - ※TypeScript の場合、as Ref<Employee[]> で型を指定しておく及安全です。
- テンプレート:
 - ⑤ HTML のテーブルの部分を確認してください。
 - ⑥ テーブルのボディ(tbody)で v-for を使用し、受け取った employeeList をループして各行を表示してください。

動作確認

- ブラウザでサイドメニューの「社員一覧」をクリックしてください。
- App.vue にあるデータが正しく受け渡され、テーブル形式で社員一覧が表示されていることを確認してください。

演習4.6. パラメータ付きルート(詳細画面の実装)

社員一覧の「氏名」をクリックした際に、その社員の「詳細画面」へ遷移するようにします。「どの社員が選ばれたか」を判断するために、URL に社員 ID を含める「パラメータ付きルート（動的ルーティング）」を使用します。



作成・修正するファイル:

- 設定: `src/router/index.ts`
- 画面: `src/views/Q4_6/EmployeeDetailView.vue` (新規作成)
- 画面: `src/views/Q4_6/EmployeeListView.vue`

a. サイドメニューのリンク先の修正 (SideMenu.vue)

`src/components/SideMenu.vue` を修正します。

RouterView to のリンク先を、Q4_5 から、全て Q4_6 に修正してください。

b. ルート定義の追加 (src/router/index.ts)

`src/router/index.ts` を編集します。

- ① 新規で追加された `EmployeeDetailView.vue` をインポートします。
- ② `routes` 配列に新しいルート定義を追加します。
 - パス: `'/Q4_6/employees/:id'` (:id が動的に変わる部分です)
 - 名前: `'employee-detail'`
 - コンポーネント: `EmployeeDetailView`

c. 一覧画面のリンク化(EmployeeListView.vue)

src/views/Q4_6/EmployeeListView.vue を編集します。

- ① テーブル内の「氏名」を表示している部分を <RouterLink> タグで囲みます。(事前に RouterLink の import が必要)
- ② to 属性には、id を含む動的なパスか、または名前付きルートを指定してください。

d. 詳細画面の作成(EmployeeDetailView.vue)

- src/views/Q4_6 フォルダ内に EmployeeDetailView.vue を新規作成しました。

Card スタイルの詳細画面です。今回の演習は、ルートからパラメータを取得する部分のみ実装します。

• スクリプト:

- ① useRoute (vue-router) をインポートしてください。
- ② useRoute() を使って、現在のルート情報を取得してください。
- ③ URL パラメータから ID を取得し、変数「id」に代入してください。※文字列として取得されるため Number() で数値変換が必要です。
 - inject で App.vue から employeeList を取得できます。
 - 受け取ったリストの中から、ID が一致する社員データを検索する処理 (find メソッド等) が行われ、詳細情報が変数に格納されます。

• テンプレート:

- 取得した社員データ(ID, 氏名, 部署, 給与など)を表示しています。

動作確認

- ブラウザで社員一覧を開き、任意の社員名 (例: 佐藤一郎) をクリックしてください。
- URL が .../employees/101 のように変化し、その社員の詳細情報が表示されることを確認してください。

演習 5 【オプション】REST API 連携による CRUD 実装

演習の目標

- Node.js と Express を用いて、Vue.js と連携する基本的 REST API サーバーを構築でき
- axios を用いた非同期通信により、データの検索・登録・更新・削除(CRUD)機能を実装できる
- ルーティングと API 連携を組み合わせ、データ操作に応じて画面が遷移する SPA を実装できる

演習の概要

第 5 章で学習した API 連携と非同期処理の手法を実践します。本演習では、あらかじめ用意されたバックエンド(ex05-api-answer)の REST API を使用して、社員情報の取得・登録・更新・削除を行います。

プロジェクト名: ex05-question

呼び出す API サーバー: ex05-api-answer (http://localhost:3000)

予想所要時間

約 60 分

演習5.1. 社員一覧の表示

src/views/EmployeeListView.vue を編集し、API から社員情報を取得して一覧表示する機能を実装してください。



作業の前提:

- Vue-router、axios モジュールは、npm から導入済みです。
- 3章で作成した Header.vue, SideMenu.vue, Footer.vue が src/components 内に存在している状態から開始します。
- REST API のソースコードは、ex05-api-answer の「server.js」を参照してください。

作成・修正するファイル:

- src/views/EmployeeListView.vue

a. Axios のインポート(EmployeeListView.vue)

- ① script setup 内で、axios をインポートしてください。

b. 全件取得関数の実装 (EmployeeListView.vue)

fetchEmployees 関数を定義し、axios.get を使用して全社員情報を取得してください。

利用 API: 1.B 全件取得 API (GET)

URL: <http://localhost:3000/api/employees/>

c. マウント時の処理 (EmployeeListView.vue)

画面初期表示時 (onMounted) に fetchEmployees 関数を実行してください。

d. 検索用変数の定義 (EmployeeListView.vue)

keyword を保持する ref 変数を定義してください (初期値は空文字)。

e. 検索機能の実装 (EmployeeListView.vue)

searchEmployees 関数を定義し、axios.get を使用して検索結果を取得してください。

利用 API: 1.A 検索 API (GET)

URL: <http://localhost:3000/api/employees/search?keyword=...>

f. 双方向バインディング (EmployeeListView.vue)

template 内で記述します。

input タグと keyword 変数を双方向データバインディングしてください。

g. リストレンダリング (EmployeeListView.vue)

template 内で記述します。

v-for を使用して employees の内容を繰り返し表示してください。

h. 詳細リンク (EmployeeListView.vue)

template 内で記述します。

RouterLink を使用して、社員詳細画面 (/employees/ID) へのリンクを作成してください。

動作確認

- API サーバーを起動します（ex05-api-answer で `node server.js`）。
- 開発サーバーを起動します（ex05-question で `npm run dev`）。
- トップページにアクセスし、社員一覧が表示されるか確認します。
- 検索ボックスに「山田」などのキーワードを入力し、「検索」ボタンをクリックして検索が動作するか確認します。
- ブラウザのアドレスバーで URL を直接書き換えて、ヘッダー等は固定されたまま、メイン部分だけが変わるか確認してください。

演習5.2. 社員詳細の表示と削除

src/views/EmployeeDetailView.vue を編集し、API から社員 1 名を取得して詳細表示する機能を実装してください。

ID	氏名	給与	勤務地
1001	山田太郎	230,000 円	東京
1002	鈴木一郎	230,000 円	大阪
1003	田中花子	300,000 円	福岡



■ 作成・修正するファイル:

src/views/EmployeeDetailView.vue

a. Axios のインポート(EmployeeDetailView.vue)

script setup 内で、axios をインポートしてください。

b. 詳細取得(EmployeeDetailView.vue)

axios.get を使用して、特定の社員情報を取得してください。

利用 API: 2. 社員詳細取得 API (GET)

URL: http://localhost:3000/api/employees/ID/

c. 削除機能(EmployeeDetailView.vue)

① axios.delete を使用して、特定の社員データを削除してください。

利用 API: 5. 社員削除 API (DELETE)

URL: http://localhost:3000/api/employees/ID/

② 削除完了後、一覧画面にリダイレクトさせるロジックも検討してください。

d. 画像表示制御(EmployeeDetailView.vue)

template 内で記述します。

image_path がある場合のみ タグを表示してください。

e. 画像パスのバインド(EmployeeDetailView.vue)

template 内で記述します。

src 属性に image_path をバインドしてください。

f. 編集リンク(EmployeeDetailView.vue)

template 内で記述します。

編集画面(/employees/edit/ID)への RouterLink を作成してください。

【動作確認 5.2】

1. 社員一覧画面から任意の社員名をクリックし、詳細画面へ遷移できるか確認します。
2. 社員情報(名前、給与、画像など)が正しく表示されているか確認します。
3. 「削除」ボタンをクリックし、確認ダイアログの後にデータが削除され、一覧画面に戻るか確認します。

演習5.3. 社員情報の編集

The diagram illustrates the process of editing employee information. It consists of three main components:

- 社員詳細 (Employee Detail):** Displays employee information including ID (1001), Name (山田太郎), Salary (230000 円), Work Location (東京), and Password (password). A red box highlights the '編集' (Edit) button.
- 社員情報編集 (Employee Information Edit):** A form for editing employee information. It includes fields for Name (山田太郎), Password (password), Salary (230000), and Work Location (東京). A red box highlights the '保存' (Save) button.
- 社員情報編集 (Employee Information Edit):** A second form showing the updated information. The Name is now '山田太郎改', Password is 'password改', Salary is '230001', and Work Location is '京都'. A red box highlights the '保存' (Save) button.

Red arrows indicate the flow from the '社員詳細' page to the first '社員情報編集' form, and from the '社員情報編集' form to the confirmation message 'localhost:5174 の内容 更新しました' (Content on localhost:5174 updated).

■ 作成・修正するファイル:

- ・ src/views/EmployeeEditView.vue

a. データ取得と初期値設定 (EmployeeEditView.vue)

- ① script setup 内で、axios をインポートしてください。
- ② axios.get を使用してデータを取得し、各 ref 変数に代入してください。

利用 API: 2. 社員詳細取得 API (GET)

URL: <http://localhost:3000/api/employees/ID/>

b. 更新処理 (EmployeeEditView.vue)

axios.put を使用してデータを更新してください。

利用 API: 4. 社員更新 API (PUT)

URL: <http://localhost:3000/api/employees/ID/>

送信データ: name, password, salary, location_name, image_path

c. フォーム送信(EmployeeEditView.vue)

button の type は、「submit」にしてください。

フォーム送信時(submit)に updateEmployee を実行するようにしてください。

d. バインディング(EmployeeEditView.vue)

template 内で記述します。

各 input タグに適切な v-model を設定してください。

【動作確認 5.3】

1. 詳細画面の「編集」リンクをクリックし、編集画面へ遷移できるか確認します。
2. フォームに既存のデータ(名前や給与など)が初期表示されているか確認します。
3. 内容を変更し、「更新」ボタンをクリックして更新が成功するか(詳細画面へ戻り、変更が反映されているか)確認します。

演習5.4. 新規社員の登録

メニュー

社員一覧

新規社員登録

新規社員登録

氏名

パスワード

給与

勤務地

写真パス (例: /images/1001.png)

※public/imagesフォルダ内のファイル名を指定してください

登録

■ 作成・修正するファイル:

src/views/NewEmployeeView.vue

a. 登録処理(NewEmployeeView.vue)

- ① script setup 内で、axios をインポートしてください。
- ② axios.post を使用して、新規社員情報を登録してください。

利用 API: 3. 社員登録 API (POST)

URL: `http://localhost:3000/api/employees/`

送信データ: `name, password, salary, location_name, image_path`

b. 画像パス(NewEmployeeView.vue)

template 内で記述します。

`image_path` 変数を `v-model` でバインドしてください。

【動作確認 5.4】

1. 社員一覧画面の「新規登録」ボタンをクリックし、登録画面へ遷移できるか確認します。
2. 各項目を入力し、「登録」ボタンをクリックして新しい社員が追加されるか確認します
3. 一覧画面に戻り、追加した社員が表示されているか確認します。

付録 A 選択式問題の解答と解説

第 1 章: Vue3 環境構築

演習 1.1. 開発環境の準備

問題: Vue.js の開発環境準備 (Node.js と VS Code) に関する記述として、誤っているものを 1 つ選んでください。

a) VS Code で複数の章 (フォルダ) をまとめて管理するために、「ワークスペース」機能を利用すると便利である。

b) Node.js をインストールすると、ブラウザ上で Java のコードが実行できるようになる。

c) VS Code には、Vue ファイルの構文ハイライトや補完を有効にするために「Vue - Official」拡張機能をインストールする。

d) Vue 3 や Vite を利用するためには、PC に Node.js (推奨版) がインストールされている必要がある。

【答え: b】

【解説】

- **b** について: Node.js は Vite や Vue の開発ツールを動かす土台となる仕組みであり、ブラウザ上で Java を実行するためのものではありません。
- **a** について: ワークスペース機能を使うことで、1 つの画面で全章のコードを見比べながら学習できるようになります。
- **c** について: VS Code に「Vue - Official」プラグインを導入することで、Vue/TS の補完やエラー検出が有効になります。
- **d** について: Vite や Vue の開発ツールは Node.js を土台に動くため、インストールが必須です。

演習 1.2. Vite でプロジェクト作成

問題: Vite を使用したプロジェクト作成の手順に関する記述として、誤っているものを選んでください。

a) コマンド `npm create vite@latest` を使用して、最新 Vite プロジェクトを作成できる。

b) コマンド実行時のディレクトリ位置 (カレントディレクトリ) にプロジェクトが展開されるため、実行場所には注意が必要である。

c) プロジェクト作成コマンドを実行した直後、自動的に依存ライブラリ (node_modules) もインストールされるため、npm install は不要である。

d) コマンドのオプションに `--template vue-ts` を指定することで、最初から TypeScript 対応の Vue プロジェクトを作成できる。

【答え: c】

【解説】

- **c** について: `create vite` コマンドでプロジェクトの雛形(テンプレート)を作成しただけでは不十分であり、動かすためには別途 `npm install` を実行してライブラリをダウンロードする必要があります。
- **a・b** について: `npm create vite@latest .` と実行することで、現在のフォルダ(ドット)に雛形を作成できます。
- **d** について: オプションに `template vue-ts` を指定することで、最初から TypeScript 対応の状態で作成されます。

演習 1.3. 依存ライブラリ導入

問題: 依存ライブラリの導入(`npm install`)と `package.json` に関する記述として、誤っているものを選んでください。

- a) `npm install` を実行すると、`package.json` の内容に基づいて必要なファイルがインターネットからダウンロードされる。
- b) ダウンロードされたライブラリの実体は、`node_modules` フォルダの中に格納される。
- c) `package.json` は、プロジェクトに必要なライブラリの名前やバージョンが記述された設計図のようなファイルである。
- d) `npm install` を実行するためには、PC がインターネットに接続されている必要はない。**

【答え:d】

【解説】

- **d** について: `npm install` はライブラリを一括「ダウンロード」する処理であるため、インターネット接続が必要です。
- **a・c** について: `package.json` に必要なライブラリが記載されており、その内容に従って適切なライブラリが導入されます。
- **b** について: インストールされたパッケージは `node_modules` フォルダに格納されます。

演習 1.4. 開発サーバーの起動と動作確認

問題: 開発サーバーの起動と動作確認に関する記述として、誤っているものを選んでください。

- a) npm run dev コマンドを実行すると、ローカル開発サーバーが起動する。
- b) VS Code の「NPM スクリプト」機能を使うと、コマンド入力をせずにマウス操作だけでサーバーを起動できる。
- c) 開発サーバー起動中は、ソースコードを編集して保存すると、ブラウザをリロードしなくても変更が即座に反映される(ホットリロード)。
- d) **開発サーバーを停止すると、作成したソースコードもすべて削除されてしまう。**

【答え:d】

【解説】

- dについて: 開発サーバーは動作確認のための仕組みであり、停止しても作成したファイル(ソースコード)が削除されることはありません。
- a・bについて: 「NPM スクリプト」メニューの dev 行にある再生ボタンをクリックすることで、Vite ローカルサーバーが起動します。
- cについて: Vite サーバーは「ホットリロード」サーバーであり、コードを保存すると即座にブラウザに反映されます。

演習 1.5. フォルダと主要ファイルの役割

問題: Vue プロジェクトのフォルダ構成と主要ファイルの役割について、誤っているものを選んでください。

- a) App.vue はルートコンポーネントと呼ばれ、画面全体のレイアウトや子コンポーネントを統括する親の役割を持つ。
- b) index.html はアプリケーションのエントリポイントであり、Vue アプリがマウントされる `<div id="app"></div>` が記述されている。
- c) **src/components フォルダには、ビルド後の成果物や画像ファイルなどの静的アセットのみを格納する。**
- d) main.ts は TypeScript のエントリポイントであり、`createApp(App).mount('#app')` を実行して Vue アプリを起動する。

【答え:c】

【解説】

- cについて: src/components は「画面の子コンポーネント(主なコーディング)」を格納する場所です。静的ファイル(画像など)は assets や public フォルダを使用します。

- **a**について: App.vue は画面の最上位(親)コンポーネントであり、子コンポーネントを呼び出すハブの役割を持ちます。
- **b**について: index.html はブラウザが最初に読み込む SPA の土台であり、表示場所となる `<div id="app"></div>` が置かれています。
- **d**について: main.ts はアプリの入口であり、ここで App.vue を #app 部分に差し込む(マウントする)処理を行います。

第 2 章:Vue の基礎知識

演習 2.1. Vuejs の特徴とメリット(クイズ)

問題: 従来の JavaScript(DOM 直接操作)の課題や、Vuejs の特徴に関する記述として、誤っているものを 1 つ選んでください。

- a) 従来の JavaScript 開発では、データの変更に合わせて手動で DOM 操作を行う必要があり、規模が大きくなるとコードが複雑化しやすいという課題があった。
- b) Vuejs は「リアクティブシステム」を備えており、JavaScript 側のデータが変更されると、自動的に画面(DOM)の表示も更新される。
- c) Vuejs はサーバーサイド(バックエンド)専用のフレームワークであり、Java や PHP のようにサーバー上で HTML を生成してブラウザに送信する仕組みである。
- d) Vuejs では、HTML・CSS・JavaScript を「単一ファイルコンポーネント(.vue)」として 1 つのファイルにまとめて管理し、部品化することができる。

【答え:c】

【解説】

- cについて: Vuejs は「フロントエンド」のフレームワークです。サーバー上で HTML を作るのではなく、ブラウザ(クライアント側)で動的に画面を制御します。
- aについて: 従来の DOM 直接操作は、動的な更新のたびに DOM 取得・更新を記述する必要があり非効率でした。
- bについて: Vue のリアクティブとは「データが変われば画面が変わる」という自動同期の仕組みを指します。
- dについて: Vue では HTML/CSS/JS を「.vue」ファイルに集約(1 部品=1 ファイル)して管理します。

第 4 章: ルーティングと SPA

演習 4.1. Vue Router の導入と役割(クイズ)

問題: Vue Router の概要と導入に関する記述として、誤っているものを選んでください。

- a) Vue Router を使用すると、URL に応じて表示するコンポーネントを切り替えることができ、SPA (Single Page Application) として動作させることができる
- b) Vue Router を既存のプロジェクトに導入する場合、ターミナルで `npm install vue-router` コマンドを実行してライブラリを追加する。
- c) Vue Router を使用するためには、`main.ts` に `.use(router)` の記述を追加し、アプリ全体でルーターを有効化する必要がある。
- d) Vue Router は Vuejs 本体に最初から組み込まれている機能であるため、別途インストールや `package.json` への追加は不要である。

【答え:d】

【解説】

- d について: Vue Router は外部ライブラリであり、導入には `npm install vue-router` によるインストールが必要です。
- a について: URL に応じてコンポーネントを切り替えることで、SPA として動作させることができます。
- c について: 作成したルーターをアプリ全体で使えるようにするには、`main.ts` で `.use(router)` と登録する必要があります。

演習 4.2. ページ（ルート）とコンポーネント（クイズ）

問題：SPA における画面構成と `<RouterView>` の役割に関する記述として、誤っているものを選んでください。

- a) SPA では、ヘッダーやサイドバーなどの共通部分は `App.vue` に固定して配置し、メインコンテンツ部分のみを切り替える構成が一般的である。
- b) `<RouterView />` は、URL に対応したコンポーネント（ページ）が表示される「場所」を確保するためのタグである。
- c) ページを切り替えるたびにブラウザが HTML ファイル全体を再読み込みするため、動作は従来の Web サイトと同じように少し遅くなる。
- d) `src/views` フォルダには、ページとして表示されるコンポーネント（例: `HomeView.vue`）を格納し、`src/components` フォルダには部品コンポーネントを格納するのが推奨される構成である。

【答え:c】

【解説】

- cについて：SPA はブラウザ全体を再読み込みするのではなく、コンポーネントのみを切り替えるため、高速でスムーズな遷移が可能です。
- a・bについて：ヘッダー等は固定し、可変領域に `<RouterView />` を配置してページを差し替えます。
- dについて：ページ単位のファイルは `src/views` フォルダに作成することが推奨されています。

Vue.js 3 フロントエンド開発入門
~作って理解する SPA(TypeScript 対応)~
演習ガイド

無断複写・複製の禁止

本書は著作権法上の保護を受けています。
本書の全部または一部を無断で複写複製することは
禁じられています。

2025 年 12 月 22 日	第 1 版	発行
2026 年 3 月 15 日	第 2 版	改訂

著 者
発行者

トレノケート株式会社
〒 163-6020
東京都新宿区西新宿 6 丁目 8 番 1 号
住友不動産新宿オークタワー 20 階
電話 03-3347-9686 FAX 03-3347-9699

<https://www.trainocate.co.jp/>